Relatório: features externas e modelos temporais no forecast de nível

Data: 2026-10-19
Branch: tasso/sector-cycle-city
Base: data/base_tratada_v2.csv (sha256 34e73afeea7ca3e4...)
Painel: nível por (setor, cidade, ciclo) - 12.103 séries, 620 setores, 31 ciclos

Este documento resume o que foi testado, os resultados, como reproduzir, e o que pode ser feito para melhorar. Ele é escrito para quem não acompanhou o trabalho.

---

1. Objetivo

Três perguntas, todas no harness compare_forecasters, ranqueadas pela mesma métrica (mae_common - MAE na interseção dos pontos que todos os candidatos preveem; é a única métrica comparável entre linhas):

1. As features externas já existentes ajudam o forecast de nível?
2. Features novas (socioeconômicas por UF; momentum; calendário) ajudam?
3. Os modelos temporais (ARIMA, ETS, cycle_factor, Prophet) batem o LightGBM?

Regra de ouro descoberta no caminho: apenas o LightGBM consome features. ARIMA, ETS, cycle_factor e Prophet operam só sobre a série de itens.

---

2. O que foi testado e os resultados

2.1 Features existentes + modelos temporais (base completa)

Run: experiments/compare_forecasters/outputs/ 2026-10-19_features_temporais

candidato	mae_common	observação
lgbm_ratio	159.6	melhor - target = correção sobre a médi
lgbm_base / regiao / gerencia / estado	165.9	categóricas de grupo não ajudam
cycle_factor:mean3	166.6	melhor temporal; quase empata o Light
ets(A,N,N)	172.7	-
naive:mean	177.0	piso
arima(1,0,0)+c	179.3	-
lgbm_companion / full	187.6 / 189.4	companion lags pioram
naive:last_value	218.3	-

Conclusão: categóricas de região/gerência/estado não movem a agulha (a série já é por setor-cidade, então a geografia já está embutida). O único ganho veio de mudar o alvo para ratio.

2.2 Features socioeconômicas por UF (base completa)

Run: experiments/compare_forecasters/outputs/ 2026-10-19_externos_uf
Candidato novo: lightgbm_external (população, densidade, salário, desemprego,

por UF, do censo 2022 + SIDRA).

candidato	mae_common
lgbm_base	159.675
lgbm_ext_socio (salário+desemprego)	159.716
lgbm_ext_todas	159.726
lgbm_ext_mercado (pop+densidade)	159.726
naive:mean	172.842

Conclusão: nenhuma variante externa ajuda (diferença ~0.05 MAE, ruído). Os dados são por UF e estáticos - a mesma informação que a categórica estado, que também não ajudou.

2.3 Momentum + calendário (base completa)

Run: experiments/compare_forecasters/outputs/ 2026-10-19_dynamics
Candidato novo: lightgbm_dynamics (momentum = lag_1 - lag_2 e lag_1 / rolling_mean_3; calendário = trimestre de abertura). Tudo sobre target: ratio.

candidato	mae_common
lgbm_base_ratio	153.604
lgbm_momentum	153.604 (idêntico)
lgbm_quarter	153.604 (idêntico)
lgbm_momentum_quarter	153.604 (idêntico)
naive:mean	172.842

Conclusão: previsões idênticas em todos os 122.067 pontos. As features são usadas nos splits (a mom_ratio_1_mean3 é a 3ª mais importante no treino), mas são colineares com os lags/rolling que o modelo já tem, então a função de decisão final não muda. Numa amostra pequena de setores as previsões diferem; na base completa, o espaço de decisão é dominado pelos lags e o efeito líquido é nulo.

2.4 Prophet (amostra de 60 setores, seed 42)

Run: experiments/compare_forecasters/outputs/ 2026-10-19_prophet_amostra
Candidato novo: prophet (por série setor-cidade, nível por ciclo).
Amostra porque Prophet na base completa seria ~133k fits (>6h). Amostra = 1.017 séries, 9.461 pontos comuns.

candidato	mae_common	bias %	n_missing
lgbm_ratio	181.1	-23.5	160
lgbm_base	183.9	-28.9	160
cycle_factor:mean3	187.8	-3.9	160
ets(A,N,N)	194.4	+9.8	596
prophet	195.0	+2.6	864
naive:mean	197.6	+15.4	160

Conclusão: Prophet ficou em penúltimo, batendo só o naive. O painel é por ciclo (~20 pontos anuais, sem sazonalidade sub-anual), fora do regime em que o Prophet brilha. Tem a pior cobertura (pula séries curtas) mas o segundo menor viés. O custo (>6h na base cheia) não justifica o uso.

3. Veredito geral

Três frentes de feature engineering (categóricas de grupo, externas por UF, derivadas de momentum/calendário) não melhoraram o forecast. O único ganho real veio de reparametrizar o alvo para ratio (prever a correção sobre a média do setor em vez do nível absoluto).

Entre os modelos, o LightGBM (com ratio) vence; cycle_factor é o melhor temporal; ETS, ARIMA e Prophet perdem. Prophet é o mais caro e um dos piores.

A razão de fundo: com ~20 ciclos de treino por série e 57,5% das linhas sendo zeros (cidades que não compraram no ciclo), o teto de ganho com features é baixo. O sinal disponível já é extraído pelos lags.

---

4. Como reproduzir

Pré-requisitos: Python 3.13, ambiente do projeto instalado (uv sync ou o .venv), e data/base_tratada_v2.csv presente. As dependências necessárias (lightgbm, statsmodels, prophet, python-calamine) já estão no pyproject.toml - o censo .xlsx é lido via engine calamine, sem openpyxl.

Cada run grava em experiments/compare_forecasters/outputs/ <run_dir>/: comparison.csv (ranking), predictions.csv (erro por linha), config.yaml e manifest.json (hash do config, hash dos dados, commit, versões dos pacotes).

```
# 2.1 features existentes + temporais (base completa, ~1h20)
python -m experiments.compare_forecasters.run \
  configs/experiments/compare_forecasters/ teste_features_e_temporais.yaml

# 2.2 externas por UF (base completa, ~25 min)
python -m experiments.compare_forecasters.run \
  configs/experiments/compare_forecasters/ teste_externos_uf.yaml

# 2.3 momentum + calendário (base completa, ~25 min)
python -m experiments.compare_forecasters.run \
  configs/experiments/compare_forecasters/ teste_dynamics.yaml

# 2.4 Prophet (amostra de 60 setores, seed 42, ~22 min)
python -m experiments.compare_forecasters.run_samp led \
  configs/experiments/compare_forecasters/ teste_prophet_amostra.yaml
```

No Windows/PowerShell, troque python por .venv\Scripts\python.exe se o uv/python não estiver no PATH.

Determinismo: o LightGBM usa random_state fixo e sem amostragem estocástica, então o MAE é idêntico bit-a-bit entre execuções. ETS/ARIMA/Prophet são igualmente determinísticos com os mesmos dados.

Amostragem (run_sampled)

O run_sampled.py aceita um bloco extra no config:

```
sample:
```

```
  n_sectors: 60
  seed: 42
```

Amostra setores inteiros (toda cidade do setor), com seed fixa, para ranquear candidatos caros (Prophet) sem rodar as 12k séries. Omitir o bloco sample roda a base completa.

---

5. Arquivos adicionados (sem alterar o LightGBM base)

Código:
- src/forecasting/external.py - loader das tabelas IBGE/SIDRA por UF.
- src/forecasting/candidates/lightgbm_exte rnal.py - candidato lightgbm_external.
- src/forecasting/candidates/lightgbm_dyna mics.py - candidato lightgbm_dynamics.
- src/forecasting/candidates/prophet.py - candidato prophet.
- experiments/compare_forecasters/run_samp led.py - runner com amostragem.

Alterações mínimas e aditivas:
- src/config/schema.py - novos *Params no union ForecastParams.
- src/forecasting/dataset.py - build_item_panel(..., external=False) + _attach_external.
- experiments/compare_forecasters/run.py - ativa external=True quando pedido.
- src/forecasting/candidates/__init__.py - registra os candidatos novos.

Configs:
- configs/experiments/compare_forecasters/ teste_features_e_temporais.yaml
- configs/experiments/compare_forecasters/ teste_externos_uf.yaml
- configs/experiments/compare_forecasters/ teste_dynamics.yaml
- configs/experiments/compare_forecasters/ teste_prophet_amostra.yaml

O src/forecasting/candidates/lightgbm.py não foi alterado.

---

6. O que fazer para melhorar

Como feature engineering não ajudou, o sinal aponta para outras direções:

1. Tratar a intermitência. 57,5% das linhas são zeros de cidades que não compraram no ciclo, e o viés é -26% (subprevisão). Modelos específicos para demanda intermitente (Croston, TSB, ou um modelo hurdle: classificar compra/não-compra e depois prever o volume condicional) atacam a maior fonte de erro diretamente.
2. Explorar mais o alvo ratio. Foi a única coisa que ganhou. Vale testar outros denominadores (mediana, mediana robusta, nível suavizado por ETS) e combinar com os melhores hiperparâmetros do LightGBM.
3. External time-varying. Se for insistir em dados externos, usar o valor do trimestre de cada ciclo (salário/desemprego mudam a cada trimestre), não a média estática. O loader external.py já tidy-fica por trimestre; falta juntar por (estado, trimestre) em vez de só estado. Expectativa moderada, dado que a versão estática não ajudou.
4. Dados city-level. Toda feature externa testada é por UF (grosseira). Um

sinal que varie *dentro* do estado (por cidade) teria chance real, já que a série é por cidade.

5. Prophet só se mudar a granularidade. Prophet no nível diário por setor-cidade (com feriados e sazonalidade semanal, como no interpol_prophet.py) joga no regime certo do modelo - mas é outro tipo de experimento, fora do ranking por ciclo.